PARALLEL & DISTRIBUTED COMPUTING ARCHITECTURE

Practice Task Sheet: Gustafson's Law & Weak Scaling

Instructor: Sufyan bin Uzayr **Assessment:** Ungraded Comprehensive Practice

Submission Target: Individual Git Repository

Focus Area: Scaled Speedup, Amdahl vs. Gustafson, Real-World Workloads

---

GIT REPOSITORY SUBMISSION GUIDELINES

All work must be authored and committed directly to your designated personal Git repository under a dedicated branch or folder named `gustafson-law-practice/`. Organize your answers logically into subdirectories: `theory/` (analytical derivations and proofs in Markdown or LaTeX), `calculations/` (worked numeric solutions), `benchmarks/` (executable scripts and driver harnesses), and `analysis/` (comparative reflections and synthesized plots).

Use clear, atomic commit messages for each task (e.g., `feat: complete task 04 derivation`). Commit raw source code, computation notebooks, and written explanations directly to your Git log.

Metric / Principle	Mathematical Formulation	Operational Definition
Scaled Speedup (Gustafson)	$S(P) = P - \alpha(P - 1) = s + p \cdot P$	Workload scales proportionally with processing units P; constant parallel execution time.
Parallel Efficiency	$E(P) = S(P) / P = 1 - \alpha \cdot ((P - 1) / P)$	Fraction of compute power effectively utilized under weak scaling conditions.
Amdahl Fixed-Size Speedup	$S_{Amdahl}(P) = 1 / (s + (1 - s) / P)$	Strong scaling baseline; constant total problem size distributed over P cores.

Part I: Theoretical Foundations & Mathematical Derivations

Strong Scaling vs. Weak Scaling Foundations — TASK 01

Contrast the foundational assumptions of Gene Amdahl (1967) and John Gustafson (1988). Explicitly define how problem dimension behaves as processor count P scales toward infinity in both models. Explain why Amdahl's Law is termed "fixed-size" scaling while Gustafson's Law represents "fixed-time" scaling.

Deliverable: Write a structured analytical summary in `theory/task01_scaling_models.md` highlighting the philosophical and physical divergence between both laws.

Formal Derivation of Scaled Speedup **TASK 02**

Derive Gustafson's scaled speedup formula $S(P) = s + p \cdot P$ from first principles. Let total parallel runtime on P cores be normalized to $s + p = 1$, where s is the serial execution time and p is the parallel execution time. Show how the equivalent serial workload is determined and prove that $S(P) = P - \alpha(P - 1)$, where α is the serial fraction.

Deliverable: Full step-by-step mathematical derivation in `theory/task02_derivation.md` with line-by-line justification.

The Serial Fraction Invariance Paradox **TASK 03**

In Amdahl's Law, the serial fraction s is measured on a uniprocessor system. In Gustafson-Barsis's Law, the fraction α (or s) is observed on the parallel machine. Demonstrate analytically why measuring the serial fraction on a parallel machine resolves the apparent contradiction where serial overhead does not throttle massive scaling.

Deliverable: A critical comparative memo in `theory/task03_serial_invariance.md` examining the frame of reference for measuring overhead.

Asymptotic Analysis of Parallel Efficiency **TASK 04**

Analyze the asymptotic behavior of parallel efficiency $E(P) = S(P) / P$ as $P \to \infty$ under Gustafson's formulation. Calculate the exact limit of $E(P)$ as a function of α. Contrast this with the limit of efficiency under Amdahl's Law over the same domain.

Deliverable: Mathematical limits, limit proofs, and explanatory deductions in `theory/task04_asymptotic_limits.md`.

Deconstructing the "Amdahl's Pessimism" Fallacy **TASK 05**

In 1988, Gustafson, Montry, and Benner demonstrated speedups exceeding 1,000× on a 1,024-processor nCUBE 10 system, which appeared impossible under Amdahl's Law with a 0.4% serial fraction. Reconstruct the historical experimental scenario and explain how problem expansion dismantled Amdahl's theoretical ceiling.

Deliverable: Case reconstruction paper in `theory/task05_historical_ncube.md` reviewing the original nCUBE experimental findings.

Part II: Mathematical Modeling & Scaled Calculations

Parametric Speedup Across Processor Densities **TASK 06**

Given an application where the parallel machine serial execution time fraction is $\alpha = 0.04$ (4%), calculate the theoretical scaled speedup $S(P)$ and parallel efficiency $E(P)$ for core topologies: $P = 16, 64, 256, 1024, 4096,$ and 16384. Tabulate and evaluate how efficiency degrades as processor count expands exponentially.

Deliverable: Complete calculation table and numeric logs in `calculations/task06_parametric_table.md`.

Reverse Engineering Maximum Tolerable Serial Overhead **TASK 07**

A high-performance supercomputing cluster consists of $P = 8{,}192$ processing cores. The operational requirement dictates that the scaled speedup must achieve at least $S(P) \geq 7{,}500$. Derive and compute the maximum allowable parallel serial fraction α_{max} that can be tolerated to maintain this performance baseline.

Deliverable: Worked algebraic derivation and exact numerical tolerance in `calculations/task07_tolerable_serial.md`.

Cross-Over Point Determination (Amdahl vs. Gustafson) **TASK 08**

Assume a workload has a static baseline serial fraction $s = 0.08$. Compute the predicted speedup using Amdahl's Law for $P = 128$. Next, compute Gustafson's scaled speedup assuming the same $\alpha = 0.08$ on 128 cores. Calculate the percentage discrepancy between the two models and explain why they diverge as P scales.

Deliverable: Step-by-step arithmetic comparisons and variance ratios in `calculations/task08_cross_divergence.md`.

Multi-Component Workload Partitioning **TASK 09**

A scientific simulation consists of three distinct computational phases: (A) Initial disk I/O and parameter broadcast taking 2% of total parallel execution time; (B) Boundary message-passing synchronization taking 6%; (C) Core domain flux computation taking 92%. Identify the aggregate serial fraction α and evaluate scaled speedup for $P = 512$ and $P = 2{,}048$.

Deliverable: Phase breakdown calculations and composite speedup analysis in `calculations/task09_multicomponent.md`.

Memory Growth Rate vs. Scaled Speedup **TASK 10**

Under pure weak scaling, problem size scales linearly with processor count: $W(P) = P \cdot W(1)$. Suppose a grid-based solver has aggregate memory footprint $M(P) = c \cdot P$. If per-node memory is strictly capped at 32 GB, calculate the maximum problem dimension N for a 3D grid (where memory scales as $O(N^3)$) across 64 and 512 nodes.

Deliverable: Mathematical formulation of memory ceilings and capacity limits in `calculations/task10_memory_growth.md`.

Part III: Comparative Scaling & Theoretical Limits

The Sun and Ni Memory-Bounded Speedup Comparison **TASK 11**

Sun and Ni introduced the Memory-Bounded Speedup model as a generalization bridging Amdahl's fixed-size and Gustafson's fixed-time models. Compare Gustafson's Law with Sun-Ni's model where workload increases with memory capacity. Identify conditions where Gustafson's Law either overestimates or underestimates achievable throughput.

Deliverable: Comparative critical essay in `theory/task11_sun_ni_comparison.md` with comparative mathematical formulations.

Communication Overhead Penalty Integration **TASK 12**

Gustafson's classic law assumes the serial fraction α remains constant as P increases. In real distributed systems, inter-node communication latency often scales as $O(\log P)$ or $O(P^{1/2})$. Formulate a modified Gustafson model that explicitly incorporates an expansion term for communication overhead $C(P)$.

Deliverable: Model extension proposal and mathematical derivation in `theory/task12_communication_penalty.md`.

Boundary Case: When Gustafson Fails **TASK 13**

Identify and evaluate two concrete parallel algorithms where Gustafson's weak scaling assumption completely fails in practice (e.g., algorithms where problem expansion changes algorithmic time complexity or triggers synchronization contention). Explain why the constant runtime premise breaks down.

Deliverable: Failure mode analysis report in `theory/task13_failure_cases.md` with detailed algorithmic breakdown.

Isoefficiency Relation with Gustafson Scaling **TASK 14**

Kumar and Grama's Isoefficiency Function measures the rate at which problem size must grow with respect to processor count P to maintain constant parallel efficiency. Re-evaluate Gustafson's scaling in terms of the isoefficiency metric and determine the isoefficiency function of a Gustafson-compliant weak scaling program.

Deliverable: Formal proof and analytical critique in `theory/task14_isoefficiency.md`.

Energy and Power Proportionality under Scaled Speedup **TASK 15**

Assuming each processor node consumes static power P_{idle} and dynamic power P_{active} during computation, formulate the energy consumption profile of a weak-scaling workload under Gustafson's model. Contrast total energy consumption when executing on 1 core versus P cores for identical wall-clock runtime.

Deliverable: Energy scaling equations and comparative analysis in `theory/task15_energy_scaling.md`.

Part IV: Domain-Specific Algorithmic Workloads

Dense Matrix Multiplication Scaling (GEMM) **TASK 16**

For dense matrix multiplication $C = A \times B$ on an $N \times N$ matrix, computational complexity is $O(N^3)$ while memory requirements scale as $O(N^2)$. Under Gustafson's assumption that parallel runtime remains constant on P processors, derive how matrix dimension N must scale with P.

Deliverable: Dimension growth rate proof and tabular scaling targets in `calculations/task16_gemm_scaling.md`.

3D Stencil Hydrodynamics Simulation **TASK 17**

In a 7-point 3D stencil finite-difference solver on a cubic domain of size $N \times N \times N$, interior updates require $O(N^3)$ operations, while boundary exchange (surface communication) scales as $O(N^2)$. Using Gustafson's Law, demonstrate why the communication-to-computation ratio improves or degrades as domain volume is scaled proportionally with P.

Deliverable: Surface-to-volume scaling proof in `calculations/task17_stencil_scaling.md`.

N-Body Gravitational Problem Scaling — TASK 18

Contrast the direct all-pairs particle method ($O(N^2)$) with the Barnes-Hut tree-code method ($O(N \log N)$). If particle count N is scaled with processor count P to maintain constant total time per iteration, determine how the serial/parallel ratio shifts under both algorithmic regimes.

Deliverable: Algorithmic comparative write-up in `analysis/task18_nbody_scaling.md`.

Monte Carlo Path Simulation: The Ideal Weak Scaling Model — TASK 19

Monte Carlo integration represents an embarrassingly parallel workload with near-zero communication. Analyze the sources of non-zero serial fraction α in large-scale Monte Carlo engines (e.g., pseudo-random number generator seed management, reduction operations, output serialization). Quantify how reduction trees impact $S(P)$.

Deliverable: Analysis document in `analysis/task19_monte_carlo.md` evaluating PRNG and tree reductions.

Parallel Graph Traversal & Irregular Workloads — TASK 20

Breadth-First Search (BFS) and PageRank on power-law graphs exhibit irregular memory access, frontier imbalance, and data-dependent synchronization. Evaluate why Gustafson's weak scaling law is notoriously difficult to achieve on graph analytics compared to regular dense linear algebra.

Deliverable: Graph scaling evaluation essay in `analysis/task20_graph_processing.md`.

Part V: Empirical Benchmarking, Simulation & Git Repository Artifacts

Parametric Speedup Simulator Implementation — TASK 21

Write a standalone simulation script in Python or C++ that models both Amdahl's Law and Gustafson's Law across arbitrary core counts ($1 \le P \le 65{,}536$) and serial fractions ($0.001 \le \alpha \le 0.20$). The tool must output structured CSV data and generate overlay plots comparing strong and weak scaling curves.

Deliverable: Commit source code to `benchmarks/simulator.py` (or `.cpp`) with automated CSV export capabilities.

Empirical Weak-Scaling Benchmark Suite — TASK 22

Implement an empirical multi-threaded benchmarking program (using OpenMP, POSIX threads, or Python multiprocessing) that executes a synthetic numerical workload (e.g., numerical integration or vector matrix transformations). Implement a weak-scaling harness where problem size per thread k remains constant while thread count varies from 1 to the host machine's maximum hardware concurrency.

Deliverable: Executable benchmark code in `benchmarks/weak_scale_bench/` with compile and run instructions.

Experimental Data Collection & Curve Fitting

TASK 23

Execute your benchmark suite from Task 22 across at least 5 distinct thread counts. Record wall-clock runtimes across 5 independent runs per configuration, calculate mean and standard deviation, and extract the empirical serial fraction $\alpha_{observed}$ using least-squares linear regression against Gustafson's formula.

Deliverable: Raw dataset in `benchmarks/results.csv` and regression fitting script in `benchmarks/fit_gustafson.py`.

Visual Plotting & Empirical vs. Theoretical Discrepancy

TASK 24

Generate publication-quality plots illustrating: (1) Ideal linear speedup; (2) Theoretical Gustafson curve based on regression fit; (3) Measured empirical data points with standard error bars; (4) Amdahl's curve calculated with the same serial fraction. Detail the hardware architecture (CPU model, cache hierarchy, memory channels) and explain deviations caused by thermal throttling or NUMA boundaries.

Deliverable: High-resolution vector plots (`plots/scaling_comparison.png` or `.pdf`) and hardware audit in `analysis/task24_hardware_audit.md`.

Executive Synthesis & Git Repository Master Documentation

TASK 25

Author the root documentation file for your practice repository (`README.md`). Synthesize all findings from Tasks 01 through 24 into an executive technical guide for parallel software engineers. Summarize core mathematical rules, provide guidelines on when to design software around Gustafson weak-scaling principles, and index every artifact submitted across your repository.

Deliverable: Master repository documentation in root `README.md` with a clean table of contents, execution commands, and architectural synthesis.